

MailTaskAgent 이해와 시연 가이드

사번 09285 백준현 AI Master 최종 제출 준비

이 문서는 개발 경험이 많지 않은 발표자가 프로젝트의 목적, 구조, 실제 동작, 기술 선택과 한계를 이해하고 직접 시연하기 위한 학습자료입니다. 결론부터 말하면 MailTaskAgent는 메일 요약기가 아니라, 메일과 기존 업무의 맥락을 관찰하고 다음 행동을 제안하며 안전 정책과 사용자 승인을 거쳐 업무 상태를 관리하는 단일 Agent 시스템입니다.

1 먼저 알아야 할 한 문장

MailTaskAgent는 새 메일을 읽고 끝나는 프로그램이 아니라 메일이 기존 업무와 어떤 관계인지 판단해 Task를 만들거나 연결하고, 회신 대기과 재개, 중요 변경 승인, 완료까지 기록하는 개인 업무관리 Agent입니다.

중요한 오해

- 한 메일이 CREATE_TASK부터 COMPLETED까지 모든 상태를 한 번에 거치는 것이 아닙니다.
- 메일 한 통마다 현재 상황에 필요한 Action 하나가 선택되고, 여러 메일과 사용자 조작이 모여 Task Lifecycle이 이어집니다.
- 같은 업무라도 WAITING_REPLY를 거치지 않을 수 있고, 완료 전 기한 변경이 없을 수도 있습니다.

2 전체 흐름

순서	무엇을 하는가	주체	확인할 결과
1	Gmail 또는 합성 Mail 입력을 표준 Schema로 변환	M-01 입력	Mail ID 방향 참여자 시각
2	업무 여부 Intent 요청사항 기반 회신 필요를 구조화	회사 LLM	MailAnalysis와 신뢰도
3	동일 Thread 우선 확인 후 필요하면 Task Mail History 검색	M-02 Python	후보 ID 점수 근거
4	관계 Target 다음 Action을 선택하고 필요 시 Query를 한 번 재작성	Task Context Agent	관계 Action 근거 신뢰도
5	Payload 상태 전이 중요 변경 후보 범위를 검사	Python Guard	ACCEPTED 또는 ASK_USER
6	승인된 Action만 SQLite에 적용	M-04 DB Tool	Task Link History
7	저장 결과를 다시 읽고 실제 결과를 관찰	Python	기대값과 DB 일치
8	업무 목록 메일 흐름 변경 기록 Trace를 표시	M-05 Streamlit	사용자가 설명 가능한 화면

3 M 01부터 M 05까지

Node	역할	입력	출력
M-01	Mail 입력 검증과 회사 LLM 의미 분석	방향 발신자 수신자 제목 본문 시각	업무 여부 Intent 요청 기한 근거 신뢰도
M-02	Task 후보 검색과 Context 구성	MailAnalysis와 기존 DB	동일 Thread 또는 top k 후보와 최근 Mail History
M-03	Agent Action 제안과 Python Safety Guard	현재 Mail 후보 상태 History	실행 Proposal 또는 ASK_USER
M-04	Task Link History Transaction과 결과 재조회	검증된 Proposal	변경 전후 값과 저장 결과
M-05	사용자 확인 Dashboard Trace	Task Mail History Event	업무 관리와 Human in the loop

M-01의 LLM과 M-03의 Task Context Agent는 모두 같은 회사 API를 사용할 수 있지만 역할과 출력 Schema가 다릅니다. M-01은 메일 의미를 구조화하고, M-03의 Agent는 검색된 후보 안에서 관계와 행동을 제안합니다.

4 Action과 Status의 차이

Action은 이번 메일을 보고 지금 수행할 행동이고, Status는 업무가 현재 놓인 상태입니다.

Action	쉬운 뜻	자동 실행 경계
CREATE_TASK	새 업무 생성	명확한 신규 요청과 필수 값이 있을 때
UPDATE_TASK	기존 업무 정보 또는 상태 변경	허용된 변경만 기한 단축은 승인
LINK_TO_TASK	변경 없이 관련 메일 연결	후보 안의 Task일 때
SET_WAITING	내가 회신해 상대 답을 기다림	OUTBOUND와 승인 발송 성공 후
MARK_COMPLETED	완료 제안	신뢰도가 높아도 사용자 승인
ASK_USER	판단 중지 후 사용자에게 이관	모호함 실패 중요 변경
IGNORE	업무가 아닌 메일 제외	공지 광고 비업무

Status	의미	다음 예
TODO	아직 시작 전	진행 시작 완료 취소

Status	의미	다음 예
IN_PROGRESS	진행 중	회신 대기 완료 취소
WAITING_REPLY	상대 답이나 자료 대기	자료 도착 후 진행 재개
COMPLETED	완료 확정	종료 상태
CANCELLED	취소 확정	종료 상태

5 왜 MailTaskAgent가 단순 Workflow가 아니라 Agentic AI인가

발표와 면접에서 가장 많이 받는 질문입니다. 답은 한 문장으로 정리됩니다. 정해진 순서는 Python이 실행하지만, 그 순서 안에서 무엇을 검색하고 어떤 행동을 할지는 입력에 따라 달라지며 그 선택을 LLM Agent가 합니다.

	고정 Workflow	MailTaskAgent
실행 순서	항상 같은 단계를 같은 순서로	M-01~M-05 뼈대는 같지만 경로가 입력에 따라 갈림
무엇을 볼지	미리 정한 필드만	관련 Task 최근 Mail History 사용자 결정을 그때그때 검색
행동 선택	if 조건문으로 열거	Agent가 후보 2~3개를 만들고 평가해 하나를 고름
틀렸을 때	그대로 실행	확신이 낮으면 검색어를 바꿔 한 번 다시 판단
안전	코드가 곧 정책	Agent 제안을 Python Guard가 따로 검증하고 위험하면 사용자에게

같은 Thread에 활성 Task가 정확히 하나면 규칙이 바로 연결하고 검색도 Agent 호출도 하지 않습니다. 반대로 다른 Thread에서 표현이 바뀐 후속 메일이 오면 검색과 Agent 판단이 돌아갑니다. 같은 프로그램인데 입력에 따라 호출 자체가 달라지는 것, 이것이 고정 Workflow와의 차이입니다.

5.1 세 용어의 쉬운 뜻

용어	쉬운 뜻	이 프로젝트에서 어디에
RAG	답하기 전에 필요한 자료를 먼저 찾아 오는 것	M-02. SQLite에서 활성 Task 최근 Mail 3건 History 5건 사용자 결정을 검색
ReAct	찾아보고 생각하고 행동하고 결과를 다시 확인하는 순환	M-02 검색부터 M-04 저장 결과 재조회까지 한 바퀴
Self-Correction	확신이 없을 때 스스로 다시 해보는 것	M-03. Query Rewrite 후 재검색 재판단 최대 1회

정확한 명칭은 SQLite Task Context RAG와 제한적 ReAct-style Self-Correction입니다. 흐름으로 쓰면 Observe에서 Retrieve로, Reason을 거쳐 Act로, 다시 Observe입니다.

아닌 것도 분명히 말합니다. Vector DB와 외부 Embedding을 쓰지 않는 Structured Retrieval이고, 단계 수에 제한이 없는 Full ReAct가 아니며, 여러 계획을 넓게 펼치는 Full Tree of Thoughts도 아니고, Agent가 여럿인 Multi-Agent도 아닙니다.

5.2 M-01부터 M-05까지 어디에 무엇이 있는가

Node	이 단계의 성격	누가 결정하는가
M-01	Observe. 메일 의미를 구조화	LLM
M-02	Retrieve. Task Context RAG 검색	Python 질의 + DB
M-03	Reason. 후보 생성 평가 선택, 필요하면 Self-Correction	LLM 두 번 호출
Guard	실행해도 되는지 검증	Python
M-04	Act와 Observe. 저장하고 저장 결과를 다시 읽음	Python + DB
M-05	사용자 확인과 Trace 표시	사용자

5.3 메일 한 통이 지나가는 길

같은 요청자에게 비슷한 업무가 두 건 있는 상황에서 후속 메일 한 통이 도착했다고 하겠습니다.

1. 현재 Mail 관찰. 발신자 제목 본문 시각을 읽고 M-01이 요청 기한 Intent를 구조화합니다.
2. SQLite Task Context RAG. 동일 Thread로 확정되지 않으므로 활성 Task 최근 Mail History 사용자 결정을 검색합니다.
3. 후보 관찰. 비슷한 Task 두 건이 후보로 올라옵니다.

4. 관계와 Action 판단. Agent가 후보 2~3개를 근거와 함께 만들고, 별도 평가 단계가 지지도를 매겨 하나를 고릅니다.
 5. 모호하면 Query Rewrite. 상위 두 지지도의 차이가 기준에 못 미치면 검색어를 바꿉니다.
 6. 재검색 재판단 1회. 다시 찾아보고 다시 판단합니다. 두 번째까지만 합니다.
 7. Python Guard. 고른 Task가 후보 안에 있는지 Intent와 Action이 맞는지 상태 전이가 허용되는지 검사합니다.
 8. 실행 또는 ASK_USER. 차이가 여전히 작으면 자동 실행하지 않고 사용자 확인으로 넘깁니다.
 9. DB 저장 결과 재조회. 실행했다면 기대한 결과와 실제 저장된 결과가 같은지 다시 읽어 확인합니다.
- 이 아홉 단계가 시연 영상의 Agent 추론 로그 구간에서 그대로 보입니다.

5.4 자주 나오는 되물음

질문	답
Agent와 Python Guard는 역할이 겹치지 않나요	겹치지 않습니다. Agent는 무엇이 맞는지 판단하고 Guard는 그 판단을 실행해도 되는지 검사합니다. Guard는 대신 판단하지 않고 막거나 사용자에게 넘기기만 합니다.
신뢰도가 높아도 완료 취소 기한 단축은 왜 승인을 받나요	되돌리기 어렵고 틀렸을 때 비용이 크기 때문입니다. 신뢰도는 모델의 자기보고 값이지 검증된 정답 확률이 아니라서 이 세 가지는 점수와 무관하게 사람이 확인합니다.
실행하고 끝내지 않고 왜 DB를 다시 읽나요	제안대로 저장되었는지는 저장해 봐야 합니다. 기대값과 실제 저장 결과를 비교하는 Observe가 있어야 ReAct의 한 바퀴가 닫힙니다.
원시 Chain-of-Thought는 왜 보여주지 않나요	모델의 날 것 사고에는 근거 없는 추측과 민감한 내용이 섞일 수 있습니다. 검증된 검색 후보와 점수 구조화된 판단 근거 Guard 결과 최종 DB 결과만 표시합니다.
그러면 Workflow는 필요 없는 건가요	반대입니다. 확정 가능한 경로와 안전 정책은 Python 규칙이 처리해야 Agent가 판단할 곳이 좁아지고 검증이 쉬워집니다. 둘을 나눈 하이브리드가 이 프로젝트의 설계입니다.

5.5 30초로 말하는 답

MailTaskAgent는 고정된 조건대로만 움직이는 자동화가 아닙니다. 현재 메일과 과거 Task Context를 검색하고, LLM Agent가 관계와 다음 Action을 판단합니다. 판단이 불확실하면 Query를 재작성해 한 번 재검색 재판단하고, Python Guard가 위험 변경을 차단합니다. 실행 후에는 DB 결과를 다시 관찰하므로 Observe Retrieve Reason Act Observe의 제한적 ReAct 구조입니다.

6 Agent와 Python과 사용자의 책임

주체	결정하거나 수행하는 것	하지 않는 것
LLM Agent	메일 의미 관계 대상 Action 회신 방식 제안	DB 직접 변경 비밀번호 출력
Python	후보 범위 Payload 상태 전이 중요 변경 검증	Agent 대신 숨겨서 다른 Action 선택
SQLite	Task Mail Link History Event Transaction 저장	업무 의미 판단
사용자	기한 단축 완료 취소 모호한 연결 실제 발송 승인	모든 일반 변경을 매번 수동 처리

7 실제 폴더와 코드 지도

위치	무엇이 있는가	발표 때 연결할 말
app.py	Streamlit 시작점	화면이 Agent Core를 호출하는 입구
src mailtaskagent workflow.py	M-01부터 M-05 오케스트레이션	전체 흐름의 중심
llm_client.py	Mail Analyzer Prompt와 구조화 재시도	메일 의미 분석
storage.py	SQLite Schema Retrieval Transaction	기억 검색 저장
task_context_agent.py	관계 Action Query Rewrite	Agentic 판단
decision.py policy.py	Action Proposal Materialize와 Guard	안전 경계
reply_assistant.py	회신 방식과 Draft	Mail to Action
approved_send_service.py	승인 발송 Guard와 결과 관찰	실제 행동
gmail_source.py	Gmail Read Send Adapter	외부 입력 출력
ui.py ui_kit.py	실제 업무 시연 Trace 화면	설명 가능한 UX
tests evidence	회귀와 실행 증거	주장과 근거 분리
Docs	AI Master 양식 구현 명세 가이드	문서와 코드 동기화

8 실제 Gmail 최종 검증 사례

메일 또는 조작	Agent와 Guard	Task 결과
신규 일정 회신 요청	초기 모델 Intent 충돌을 Guard가 ASK_USER로 차단 후 사용자 생성	TASK-009 TODO 기한 9월 16일
앱 승인 회신	DATE_REPLY 0.95 날짜 입력 수신자 Thread 잠금	발송 성공 후 WAITING_REPLY
기한 9월 15일로 단축	DUE_DATE_CHANGE 0.95 동일 Thread 중요 변경 승인	승인 후 기한만 변경
점검 대상 자료 도착	INFORMATION_RECEIVED	IN_PROGRESS로 재개
완료 처리 요청	COMPLETION 0.95이나 자동 완료 금지	승인 후 COMPLETED
같은 메일함 재동기화	33건 모두 중복 판정	Task와 History 증가 없음

실패에서 배운 점

Prompt에 WAITING은 OUTBOUND라고 써도 LLM이 항상 지킨다고 가정할 수 없었습니다. 그래서 INBOUND와 WAITING 조합을 의미 계약 오류로 검사하고 최대 한 번 재질의하도록 보강했습니다. Guard를 약화하지 않았고, 동등 입력의 회사 LLM 재검사는 NEW_TASK와 기한 9월 16일을 반환했습니다.

9 화면 사용법

메뉴	사용자 목적	시연 포인트
업무 홈	오늘 우선업무와 상태 확인	자동 갱신 Gmail 상태 검토 대기
내 업무	검색 상태 필터 상세 완료	우선순위 근거와 상세 보기
업무 상세 메일 흐름	수신 발신 본문 참여자 확인	각 행 클릭 한 통만 표시
AI 회신 준비	방식 판단 입력 Draft 승인 발송	신뢰도와 안전 정책
변경 기록	Agent 제안과 사용자 결정 확인	전후 값과 History
검토 요청	중요하거나 모호한 제안 확정	승인 전 DB 미변경
운영 상태	Scheduler Sync 실패 상태 확인	사용자 업무 화면과 운영 모니터링 분리
MVP 시연 검증	격리 DB와 반복 Case 검증	실 Gmail 본편 뒤 증거로 짧게

10 발표와 시연 순서

1. 문제 30초: 메일 요청이 후속 메일 속에서 바뀌고 누락되는 문제를 설명합니다.
2. 구조 90초: M-01부터 M-05, Agent Proposal과 Python Guard의 역할을 설명합니다.
3. 실 Gmail 2분: 신규 Task, 메일 흐름, 회신 준비, 사용자 승인과 상태 변화를 보여줍니다.
4. Agentic 근거 60초: RAG 후보, 신뢰도, Query Rewrite, Trace와 Human in the loop를 보여줍니다.
5. 검증 30초: 179 passed, Live 15/15와 28/28, 실제 Gmail 5-message E2E를 구분해 말합니다.
6. 한계 20초: Gmail 테스트 계정과 Plain Text 단일 수신자까지이며 Outlook 운영은 Post MVP라고 말합니다.

시연 중 매 화면에서 반드시 말할 세 문장

- 지금 어떤 입력이 들어왔는가
- Agent와 Python이 각각 무엇을 했는가
- 어떤 화면이나 DB 결과를 보면 성공이라고 판단할 수 있는가

11 면접 핵심 답변

질문	자기 말로 답할 핵심
왜 Agentic AI인가	Workflow는 실행 틀입니다. 확정 경로는 규칙으로 처리하고 열거하기 어려운 Task 관계 Action 회신 방식은 Agent가 Context를 검색해 동적으로 판단하며 결과 관찰과 사용자 이관까지 이어집니다.
왜 Python Rule도 쓰나	LLM은 유연한 의미 판단에 쓰고, 상태 전이와 중요 변경 같은 안전 책임은 재현 가능한 Python이 맡습니다.
RAG는 무엇을 검색하나	외부 문서가 아니라 SQLite의 활성 Task 최근 Mail History 사용자 결정을 top k로 제한 검색합니다.
왜 재시도는 한 번인가	무한 Loop와 비용 지연을 막으면서 표현 차이로 놓친 후보를 한 번 보완하고 실패하면 사람에게 넘기기 위해서입니다.
신뢰도 0.95면 왜 승인하나	자기보고 수치일 뿐 정확도 확률이 아니며 완료 취소 기한 단축 발송은 결과 영향이 커 정책상 승인합니다.
실패 경험은	INBOUND 요청을 WAITING으로 반환한 실제 Case를 Guard가 막았고 의미 계약 검증과 재질의 테스트를 추가했습니다.
테스트 수치의 한계는	179는 코드 회귀 범위, 15/15와 28/28은 정의된 합성 Ground Truth, Gmail E2E는 승인된 테스트 Thread 범위입니다.
Outlook 전환에는	Graph App Registration 사내 인증 권한 서버 운영 DB Event Subscription 보안 검토가 필요합니다.

12 현재 완료와 남은 제출 작업

구분	상태	근거 또는 다음 작업
MVP 기능	완료	RAG ReAct Action Proposal Guard HITL Trace Gmail 승인 발송
자동 회귀	완료	179 passed
회사 LLM	완료	15/15 Case 28/28 Action
실 Gmail	완료	5-message Lifecycle 중복 재조회 33/33
문서 동기화	완료	IMPLEMENTATION과 AI MASTER 공식 구조 유지
발표자료 영상	진행	4장 PDF와 5분 이하 MP4 제작 검수
사용자 리허설 면접	남음	직접 조작 1회와 질문 5개 모의면접
Outlook 사내 운영	범위 밖	Post MVP

최종 판단은 제출 범위에서 알려진 차단 오류가 없다는 뜻입니다. 모든 자연어 메일과 사내 Outlook 운영에서 오류가 없다는 뜻은 아닙니다. 이 한계를 정확히 말하는 것이 기술적 신뢰도를 높입니다.